

Question description

There are three common ways to traverse the nodes of a binary tree:

- *Preorder*: First process the root, then the left subtree, and finally the right subtree.
- *Inorder*: First process the left subtree, then the root, and finally the right subtree.
- *Postorder*: First process the left subtree, then the right subtree, and finally the root.

There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal.

Constraints

- $1 \leq n \leq 10^5$

Input

The first input line has an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers.

You can assume that the input corresponds to a binary tree.

Output

Print the postorder traversal of the tree.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 3 2 1 4
3 5 1 2 4

EXPECTED OUTPUT

3 1 4 2 5

Test Case 2

INPUT (STDIN)

6
6 5 3 2 1 4
6 3 5 1 2 4

EXPECTED OUTPUT

3 1 4 2 5 6

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=1;i<=n;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

390

Test Case 3

NLOC

41

```
Tree12.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 100005
5
6 int pre[MAXN];
7 int in[MAXN];
8 int pos[MAXN];
9
10 void solve(int pre_l, int in_l, int len) {
11     if (len <= 0) return;
12     int root = pre[pre_l];
13     int root_pos = pos[root];
14     int left_len = root_pos - in_l;
15     int right_len = len - 1 - left_len;
16
17     solve(pre_l + 1, in_l, left_len);
18     solve(pre_l + 1 + left_len, root_pos + 1, right_len);
19
20 }
21
22 int main() {
23     int n, i;
24     if (scanf("%d", &n) != 1) return 0;
25     for(i=1;i<=n;i++) {
```

main | CH.SC.U4CSE25225 | Tree12.c

Ln 36, Col 1 Spaces: 4 UTF-8 CRLF C

```
26     scanf("%d", &pre[i]);
27 }
28 for(i=1;i<=n;i++) {
29     scanf("%d", &in[i]);
30     pos[in[i]] = i;
31 }
```

```
32 solve(1, 1, n);  
33 printf("\n");  
34 return 0;  
35}  
36
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

5 3 2 1 4

3 5 1 2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree12.exe

3 1 4 2 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

6

6 5 3 2 1 4

6 3 5 1 2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree12.exe

3 1 4 2 5 6

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>